

Final Exam Study Guide

Contents

- Chapter 8 — Decrease-by-Half (Divide and Conquer)
- Chapter 9 — Randomization
- Chapter 10 — Reduction
- Chapter 11 — Dynamic Programming
- Chapter 12 — Lower Bounds
- Chapter 13 — Easy and Impossible Problems
- Chapter 14 — NP-Hard / NP-Completeness
- Parallel Algorithms
- Quick-Reference Table · Exam Questions · Study Strategy

Final Exam Study Guide — Chapters 8-14 + Parallel Algorithms

110-minute limit · one page of notes allowed · non-cumulative (covers Ch 8-14 only) · bring your CWID

[Slides] marks material taken directly from Kevin A. Wortman lecture slides. Struck-through topics are **not on the exam**: American Change Making; Ch. 12.6 Reduction Arguments; **parallel merge** details. You will **not** be asked to prove an NP-complete reduction.

Chapter 8 — Decrease-by-Half (Divide and Conquer)

8.1 The Big Idea

Solve a problem by reducing it to a smaller instance of the same problem, then combining the sub-solution(s) into a full solution. Decrease-by-half specifically shrinks the input by a **constant fraction** each step (not just by 1, as in decrease-by-one patterns like insertion sort).

[Slides] General template:

```
decrease_by_half(INPUT):
    if INPUT is base case:
        return base case solution
    left_input, right_input = divide INPUT in half
    left_solution = decrease_by_half(left_input)
    right_solution = decrease_by_half(right_input)
    entire_solution = combine left_solution with right_solution
    return entire_solution
```

Three phases every time: **Divide → Recursion → Combine**.

[Slides] Formal definition of algorithm (Def. 6): a process that, given a problem instance, produces a solution, and is (1) clear enough to implement, (2) correct, and (3) terminates (finite time).

[Slides] **Pitfall — infinite recursion**: base cases and combine must work together so recursion bottoms out for every n . Best practice: general-case threshold $t = 2$ (base cases handle $n = 0$ and $n = 1$; general case $n \geq 2$). Missing $n = 1$ causes infinite recursion — e.g. `dbh_sum([7])` splits into `[]` and `[7]` forever.

8.2 Example: Summation

Split the array in half, recursively sum each half, add the results.

```
dbh_sum(L):
    n = len(L)
    if n == 0: return 0
    if n == 1: return L[0]
    half = n / 2
    left = L[:half]
    right = L[half:]
    return dbh_sum(left) + dbh_sum(right)
```

[Slides] Dividing: `half = n/2` (integer truncation); each slice copy costs $n/2$ steps. Primitive = 1 step; list of k elements = k steps.

[Slides] Reality check: `dbh_sum` is **$O(n \log n)$** ; naïve iterative sum is **$O(n)$** and simpler. Decrease-by-half is not automatically best — compare against naïve first.

8.3 Analyzing Decrease-by-Fraction Algorithms — Master Method

[Slides] Master-form recurrence:

```
T(n) = r · T(n/d) + c(n)    ∀ n ≥ t
T(n) ∈ O(1)              ∀ n < t
where c(n) ∈ O(n^k); r, t ≥ 1; d > 1; k ≥ 0
```

Symbol	Meaning	Constraint
r	number of recursive calls	$r \geq 1$
d	divisor of input size	$d > 1$
t	general-case threshold	$t \geq 1$
k	exponent in non-recursive work $c(n)$	$k \geq 0$

[Slides] Master Theorem:

1. If $r < d^k \rightarrow T(n) \in O(n^k)$ (case 1)
2. If $r = d^k \rightarrow T(n) \in O(n^k \log n)$ (case 2)
3. If $r > d^k \rightarrow T(n) \in O(n^{\log_d r})$ (case 3)

[Slides] 6-step Master Method (memorize):

1. Identify threshold t (best practice: $t = 2$)
2. Analyze base case ($n < t$) $\rightarrow$ usually $O(1)$
3. Step-count general case $\rightarrow$ derive $T(n) = r \cdot T(n/d) + c(n)$
4. Identify/check parts: $c(n), r, d, t, k$
5. Compare r vs $d^k \rightarrow$ pick case
6. Simplify and conclude (e.g. $O(n \log n)$)

[Slides] Worked `dbh_sum`: $t=2$; $T(n)=2T(n/2)+n+4$; $r=2, d=2, k=1 \rightarrow 2=2 \rightarrow$ case 2 $\rightarrow O(n \log n)$.

8.4 Merge Sort

[Slides] Sorting problem: input list U of comparable elements; output S in non-decreasing order. Lists of size ≤ 1 are vacuously sorted.

```
merge_sort(U):
    if n ≤ 1: return U
    half = n / 2
    left = U[:half]; right = U[half:]
    return merge(merge_sort(left), merge_sort(right))

merge(LS, RS):
    S = []; li = 0; ri = 0
    while li < len(LS) and ri < len(RS):
        if LS[li] ≤ RS[ri]: S.add_back(LS[li]); li += 1
        else: S.add_back(RS[ri]); ri += 1
    return S + LS[li:] + RS[ri:]
```

Merge is $O(n)$ (two-finger; fronts are always the least remaining). Master method on merge_sort: $T(n)=2T(n/2)+O(n) \rightarrow$ case 2 $\rightarrow O(n \log n)$. Be able to **trace** splits and merges. Selection Sort $O(n^2)$ is obsolete — prefer Merge Sort or another $O(n \log n)$ sort.

[Slides] **Binary Search** (third worked decrease-by-half example; also a pre-sorting / reduction preview): only recurse into one half $\rightarrow r=1, d=2, k=0 \rightarrow$ case 2 $\rightarrow O(\log n)$.

```
binary_search(L, s, e, q):
    n = e - s
    if n == 0: return None
    if n == 1: return s if L[s]==q else None
    half = s + (n / 2)
    if q == L[half]: return half
    if q < L[half]: return binary_search(L, s, half, q)
    return binary_search(L, half + 1, e, q)
```

Chapter 9 — Randomization

9.1 The Big Idea

Randomization = intentionally making random choices. Deterministic = no random behavior. Helps when (1) many alternatives, (2) most are good, (3) picking the best deterministically would be slow.

9.2 Generating Random Numbers

[Slides] TRNG = physical entropy; PRNG = deterministic formula that looks random (default “random”).

Operation	Pseudocode	Time
Random int in $[min, max]$	<code>random_int(min, max)</code>	$O(1)$
Random element of L	<code>random_choice(L)</code>	$O(1)$
Shuffle L	<code>random_shuffle(L)</code>	$O(n)$

Expected time: $E[X]$ = weighted average of outcomes. “ $O(f(n))$ expected” means $E[\text{runtime}] \in O(f(n))$. Preference: deterministic > amortized > expected. “Expected” is sticky: $O(x)$ expected + $O(y)$ det. = $O(x+y)$ expected.

9.4 The Las Vegas Pattern

Las Vegas = always returns a **correct** answer; randomness only affects **runtime**. Randomized quick sort is Las Vegas: always sorts correctly; expected $O(n \log n)$, worst-case $O(n^2)$.

9.5 Out-of-Place Quick Sort

Divide by **value** (pivot/partition), not by index. Partition into less / equal / greater. Equal never needs sorting. Combine = concatenate (not merge).

```
quick_sort(U):
    if n ≤ 1: return U
    pivot = random_choice(U)
    less = [x for x in U if x < pivot]
    equal = [x for x in U if x == pivot]
    greater = [x for x in U if x > pivot]
    return quick_sort(less) + equal + quick_sort(greater)
```

Deterministic extreme pivots $\rightarrow T(n)=T(n-1)+O(n) \rightarrow O(n^2)$. Random pivot $\rightarrow O(n \log n)$ expected, $O(n^2)$ worst-case. Be able to **trace by hand**.

9.6 In-Place Quick Sort

Sort within index range $[s,e]$ using swaps. Random pivot swapped to end; grow $<$ zone from left (i) and $\geq$ zone from right (j); final swap places pivot. Partition $O(n)$. Same time as out-of-place: **$O(n \log n)$ expected / $O(n^2)$ worst**; better space.

Algorithm	Expected	Worst-case	In-place?
Selection Sort	$O(n^2)$	$O(n^2)$	yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	no
Quick Sort	$O(n \log n)$	$O(n^2)$	yes

Never use $O(n^2)$ sorts. Default Quick Sort; use Merge Sort when worst-case $O(n^2)$ is unacceptable.

Chapter 10 — Reduction

10.1 The Big Idea

[Slides] Design an algorithm that uses an off-the-shelf algorithm or data structure for the hard part. Template: preprocess $\rightarrow$ solve B $\rightarrow$ postprocess. If A reduces to B, A is easier than (or tied with) B. Informal analysis (after midterm): cite bottleneck big-O only.

10.2 Reduction to Sorting

Median finding: sort, pick middle $\rightarrow O(n \log n)$ expected.

[Slides] Set intersection via sort: `quick_sort(L+R)`, then take adjacent duplicates $\rightarrow O(n \log n)$ expected.

[Slides] Min-and-max via sort: front/back of sorted list $\rightarrow O(n \log n)$ expected (linear scan is faster in practice; this is a reduction example).

10.3 Reduction to Hash Table Operations

[Slides] Map stores associations (key $\rightarrow$ value). Hashing: key $\rightarrow$ hash code $\rightarrow$ array index. Collisions inevitable (pigeonhole); chaining or open addressing. Critical ops: **$O(1)$ expected**.

Pattern: find $O(n)$ search/count bottleneck inside an $O(n)$ loop $\rightarrow$ replace with hash ops $\rightarrow O(n^2) \rightarrow O(n)$ expected.

```
set_intersect_hash(L, R):
    hm = HashMap()
    for r in R: hm[r] = True
    S = []
    for x in L:
        if x in hm: S.add(x)
    return S
```

Mode average: count frequencies in a map, then find max $\rightarrow O(n)$ expected.

10.4 Priority Search Queues; Prim-Jarník and Dijkstra

[Slides] PSQ: Insert $O(1)$, RemoveMin $O(\log n)$ amortized, DecreasePriority $O(1)$ amortized. Speeds Prim-Jarník (MST) and Dijkstra (NNSSSP) to **$O(m + n \log n)$** (without PSQ: $O(mn)$).

Chapter 11 — Dynamic Programming

11.1 The Big Idea

[Slides] **Tabulation:** store instance solutions in a table. Base cases initialized immediately; general cases from other table cells; initialize all cells; answer lives in the table. Top-down = recursion/DBH; bottom-up = DP.

Design process: dimensions $\rightarrow$ **table invariant** $\rightarrow$ base $\rightarrow$ general $\rightarrow$ bottom-up draft.

```
dynamic_programming_1D(input):
    A = array large enough
    Initialize base-case elements of A
    for i from smallest to largest general case:
        A[i] = initialize using A[<i]
    return A[solution index]
```

11.2 1D DP and Fibonacci — $O(n)$

$F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$. Naïve recursion is exponential.

[Slides] Invariant: $A[i] = F(i)$.

```
fibonacci_dynamic_programming(n):
    A = [0] * (n + 1)
    A[1] = 1
    for i from 2 to n:
        A[i] = A[i-1] + A[i-2]
    return A[n]
```

$O(n)$. Be able to trace table A (demo: $n=5 \rightarrow$ output 5) and design similar 1D DP.

~~11.3 American Change Making — not on exam.~~

Edit Distance — $O(|s| \times |t|)$ / $O(n^2)$

[Slides] Min overwrite/insert/delete ops to transform start s into goal t (Levenshtein). 2D because you can shrink s or t .

Invariant: $A[i][j]$ = edit distance between $s[:i]$ and $t[:j]$.

Base: $A[0][j]=j$ (j inserts); $A[i][0]=i$ (i deletes).

```
if s[i-1] == t[j-1]:
    A[i][j] = A[i-1][j-1]          # match
else:
    A[i][j] = min(1 + A[i-1][j],   # delete
                  1 + A[i][j-1],   # insert
                  1 + A[i-1][j-1]) # overwrite
# answer at A[s.size][t.size]
```

Nested loops $\rightarrow O(|s| \times |t|)$. Trace demo: $s="has", t="make" \rightarrow$ distance 3.

Chapter 12 — Lower Bounds

12.1 The Big Idea

[Slides] Act II: prove aspects of problems, not algorithms. Lower bound = “speed limit.” NP-complete $\approx$ only exp. exhaustive search seems to work. Undecidable = impossible.

12.2 Notation and Terminology

- **Upper bound O** for a problem: exhibit an algorithm (construction).
- **Lower bound Ω** : every algorithm takes at least that long.
- **Tight bound Θ** : upper and lower match $\rightarrow$ “solved science.”

12.3 Proving Negatives

Proving a positive is easy (show an example). Proving “no algorithm is faster...” is hard — that is why lower-bound proofs feel different.

12.4 Trivial Lower Bounds

[Slides] Strategy: any correct algorithm must create the output $\rightarrow$ time $\geq$ output size. Merge output has n elements $\Rightarrow \Omega(n)$. With merge’s $O(n)$ algo $\Rightarrow$ Merge is **$\Theta(n)$** . Sorting trivial LB is only $\Omega(n)$ — leaves a gap under $O(n \log n)$.

12.5 The Tight Bound for Sorting

[Slides] Comparison-based sorting: $n!$ permutations; each comparison halves candidates; need $\log_2(n!) = \sum \log_2 i$ comparisons; n terms that are $\Omega(\log n) \Rightarrow \Omega(n \log n)$. Merge sort gives $O(n \log n)$ upper bound $\Rightarrow$ **$\Theta(n \log n)$** tight. No comparison sort can be asymptotically faster.

~~12.6 Reduction Arguments — not on exam.~~

Chapter 13 — Easy and Impossible Problems

13.1 / 13.2 Efficiently-Solvable Problems and P

[Slides] Complexity class = set of problems. **P** = solvable in polynomial time = $O(n^k)$ for some $k \geq 0$. Includes $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, ... Not poly: $O(2^n)$, $O(n!)$.

Prove $X \in P$: exhibit any poly-time algorithm (naïve OK) + analyze. Example: Pair Sum double loop $O(n^2) \Rightarrow \in P$.

13.3 Unsolvable Problems and Decidability

Decidable = some algorithm solves it (any finite time). Undecidable = no algorithm exists. $P \subseteq$ Decidable.

[Slides] **Halting Problem**: input (proc, I); True iff proc(I) halts. Undecidable by contradiction: assume `halts` $\rightarrow$ `self_halts` $\rightarrow$ `contrary` $\rightarrow$ `contrary(contrary)` both cases contradict. Compilers cannot detect all infinite loops.

Chapter 14 — NP-Hard / NP-Completeness

14.1 Verifiable Problems and NP

[Slides] **NP** = exhaustive search with a poly-time verifier. Solve = produce output; verify = check one candidate. Prove $\in$ NP: define candidate + poly verifier. $P \subseteq$ NP. Clique decision: candidate = vertex set C ; check edges — poly $\Rightarrow \in$ NP.

14.2 Hard Problems and NP-Hardness

[Slides] $E \leq_p H$ = poly-time reduction solving E via H (H harder or tied). H is **NP-hard** if $\forall E \in \text{NP}, E \leq_p H$. **NP-complete** = $\in$ NP and NP-hard (Goldilocks: not impossible + not easy).

14.3 A First NP-Complete Problem

[Slides] **Circuit Satisfaction (CSAT)**: input Boolean circuit C (DAG of AND/OR/NOT/OUT/vars); output satisfying assignment or None. Cook-Levin: CSAT is NP-complete. (1) $\in$ NP: evaluate circuit on candidate assignment, $O(n^2)$. (2) NP-hard: compile any NP verifier into a circuit; solving CSAT recovers a solution for arbitrary $E \in \text{NP}$.

14.4 Proving NP-Completeness by Reduction

Once one NPC exists: pick known NPC E , reduce $E \rightarrow H$ with poly pre/post, show $H \in \text{NP}$. **You will NOT prove an NPC reduction on the exam** — know the idea and categorize problems.

Notable NPC: CSAT, SAT, Clique, Vertex Cover, TSP, Set Partition, many generalized games.

14.5 P Versus NP Revisited

Know $P \subseteq \text{NP}$; open whether $P \subset \text{NP}$ or $P = \text{NP}$ (\$1M Clay). If any NPC is in $P \Rightarrow P = \text{NP}$ (all NP get poly algos; crypto breaks). Conjecture/advice: live as if $P \subset \text{NP}$ — treat NPC as impractical.

Class	Meaning	Example
P	poly-time solvable	Sorting, Edit Distance, Fib, Pair Sum
NP-complete	in NP + NP-hard	CSAT, SAT, Clique, Vertex Cover, TSP
Undecidable	no algorithm exists	Halting Problem

Parallel Algorithms

Overcoming Lower Bounds

[Slides] Sequential lower bounds still hold for total work, but parallel algorithms can reduce user wait time below the sequential bound.

Analyzing Parallel Algorithms — p , Work, Span

- p = number of processors
- **Work** = total steps by all processors (ignore parallel; count as usual)
- **Span** = elapsed time start→finish (take **max** of parallel branches)

```
in parallel:
  statement 1
  statement 2
```

Goal: work matches lower bound; span beats it. Scalable: span $O(X/p)$ for LB $\Omega(X)$. Embarrassingly parallel: span constant in n .

Parallel Sorting — $O(n \log n)$ work, sublinear span

Parallel merge sort: recurse on halves in parallel. ~~Parallel merge itself is NOT on the exam~~, but know overall: work $O(n \log n)$, span $O(\log^3 n)$.

Samplesort: oversample to pick $p-1$ splitters; bucket elements in parallel; recursively sort buckets; concatenate. Work $O(n \log n)$ **expected**; span $O(\log n)$ **expected**. Better span than parallel merge sort, but expected/randomized.

Quick-Reference Table of Efficiency Classes

Topic	Time Complexity
Merge Sort	$\Theta(n \log n)$ / $O(n \log n)$
Binary Search	$O(\log n)$
Out-of-Place / In-Place Quick Sort	$O(n \log n)$ expected / $O(n^2)$ worst
Reduction to Sorting (median, set n)	$O(n \log n)$
Hash table critical ops	$O(1)$ expected
Naive → hash-sped-up	$O(n^2) \rightarrow O(n)$ expected
Prim-Jarník / Dijkstra + PSQ	$O(m + n \log n)$
1D DP (Fibonacci)	$O(n)$
Edit Distance (2D DP)	$O(n^2)$ / $O(s \times t)$
Comparison sorting lower bound	$\Omega(n \log n)$, tight $\Theta(n \log n)$
Parallel merge sort	work $O(n \log n)$, span $O(\log^3 n)$
Samplesort	work $O(n \log n)$ exp, span $O(\log n)$ exp

Types of Exam Questions to Expect

- Define terminology in your own words
- Write a problem definition
- Perform a step count
- Prove an efficiency class (limits or properties of O)
- Analyze pseudocode (step count + efficiency proof together)
- Trace algorithm execution by hand, showing work
- Design algorithms using decrease-by-half, reduction, or DP patterns
- Justify / prove a lower bound
- Categorize a problem as P, NP-complete, or undecidable
- Prove a problem is in P or in NP

NOT required: proving an NP-complete reduction; the parallel merge step; American Change-Making; Reduction Arguments (12.6).

Study Strategy

- Redo Problem Sets from Decrease-By-Half onward — exam questions mirror that style.
- For each algorithm practice: (1) hand trace, (2) recurrence/step count, (3) efficiency class with justification.
- Memorize the **6-step Master Method** cold. **[Slides]**
- Practice the $\Omega(n \log n)$ sorting lower bound proof (info-theoretic / decision-tree style).
- Explain in plain language: P vs NP vs NP-hard vs NP-complete vs undecidable.
- **[Slides]** Hand-trace dbh_sum, merge_sort (+ merge), binary_search, quick sort, fib table, edit-distance grid.
- Compress essentials onto your **one allowed page** of notes (both sides OK); write your name on it.